

AI Planning

Planning Representation.



Eva Onaindia

Universitat Politècnica de València

Acknowledgements

Most of the slides used in this course are taken or are modifications from Dana Nau's lecture slides for the textbook *Automated Planning*, licensed under the Creative Commons Attribution-NonCommercial-ShareAlike License:

<http://creativecommons.org/licenses/by-nc-sa/2.0/>

<http://creativecommons.org/licenses/by/3.0/es/>

I would like to gratefully acknowledge Dana Nau's contributions and thank him for generously permitting me to use aspects of his presentation material.

Outline.

- Planning representation
- Classical representation (ex. DWR and blocks world)
- State-variable representation (ex. DWR and blocks world)
- Comparisons
- PDDL: Planning Domain Description Language

Quick Review of Classical Planning

- Classical planning requires all eight of the restrictive assumptions:

A0: Finite

A1: Fully observable

A2: Deterministic

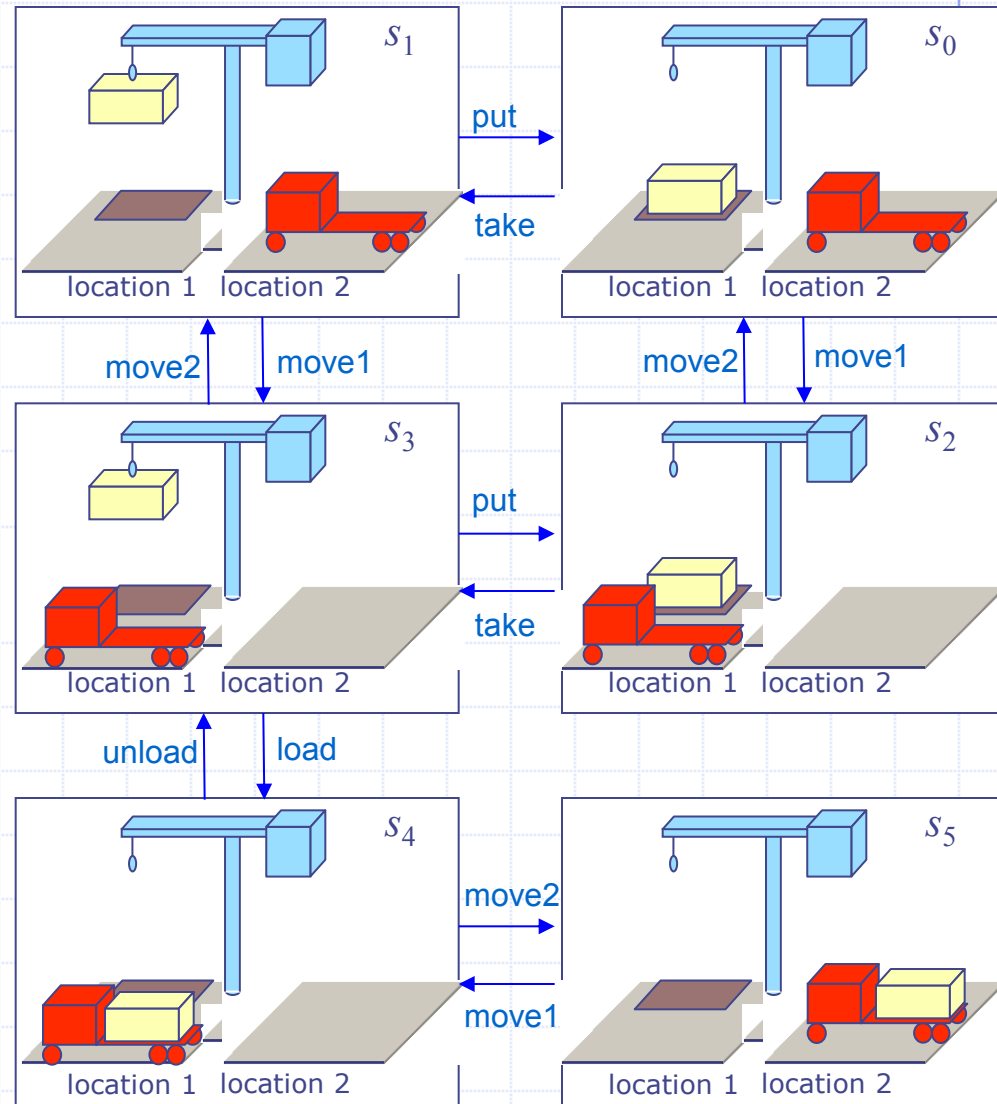
A3: Static

A4: Attainment goals

A5: Sequential plans

A6: Implicit time

A7: Offline planning

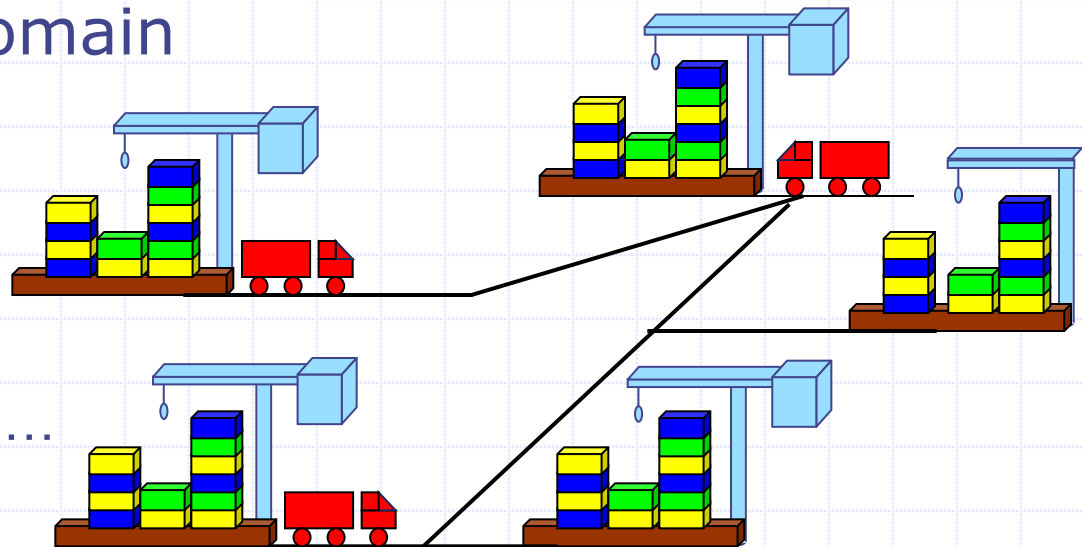


Planning representation. Motivation.

- In most problems, far too many states to try to represent all of them explicitly as $s_0, s_1, s_2, \dots$
- Represent each state as a set of features
 - e.g.,
 - a vector of values for a set of variables
 - a set of ground atoms in some first-order language L
- Define a set of *operators* that can be used to compute state-transitions
- Don't give all of the states explicitly
 - Just give the initial state
 - Use the operators to generate the other states as needed

Classical Representation

- Start with a first-order language
 - Language of first-order logic
 - Restrict it to be *function-free*
 - Finitely many predicate symbols and constant symbols, but *no* function symbols
- Example: the DWR domain
 - Locations: loc1, loc2, ...
 - Containers: c1, c2, ...
 - Pallet: p1, p2, ...
 - Robot carts: r1, r2, ...
 - Cranes: crane1, crane2, ...



Classical Representation

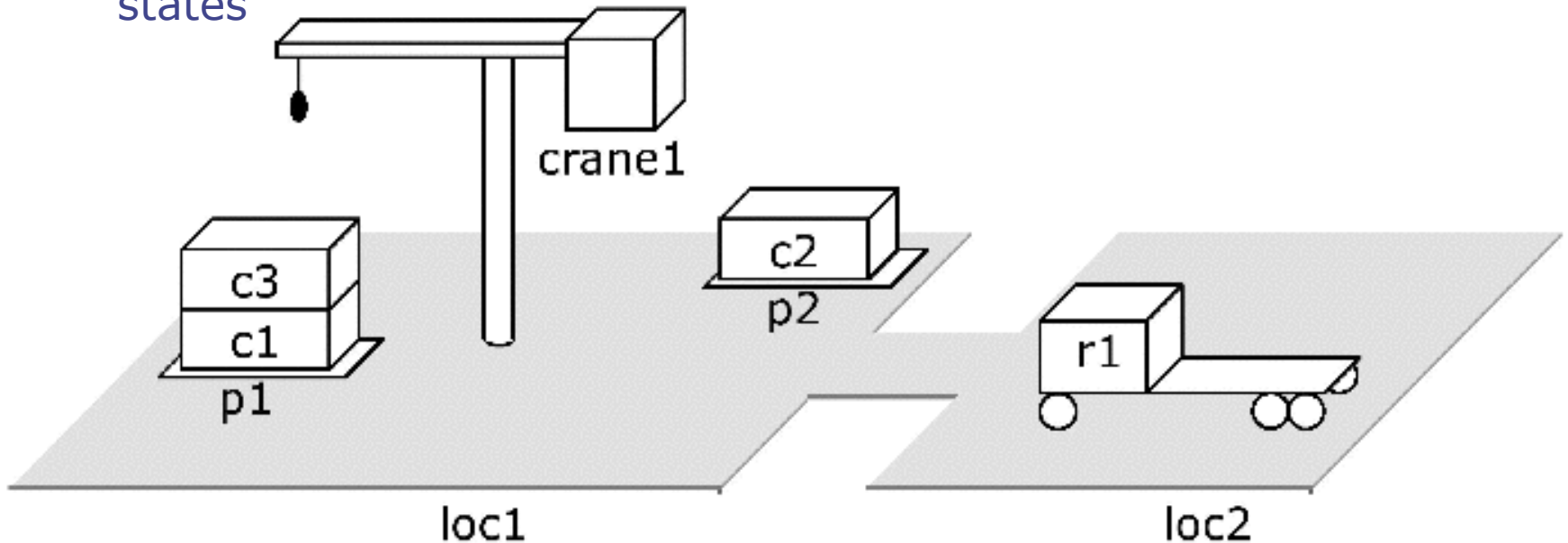
- *Atom*: predicate symbol and args
 - Use these to represent both fixed and dynamic relations

adjacent(<i>l, l'</i>)	attached(<i>p, l</i>)
occupied(<i>l</i>)	at(<i>r, l</i>)
loaded(<i>r, c</i>)	unloaded(<i>r</i>)
holding(<i>k, c</i>)	empty(<i>k</i>)
in(<i>c, p</i>)	on(<i>c, c'</i>)
top(<i>c, p</i>)	belong(<i>k, l</i>)

- *Ground* expression: contains no variable symbols - e.g.,
in(c1,p3)
- *Unground* expression: at least one variable symbol - e.g.,
in(c1,x)
- *Substitution*: $\theta = \{X_1 \leftarrow V_1, X_2 \leftarrow V_2, \dots, X_n \leftarrow V_n\}$
 - Each x_i is a variable symbol; each v_i is a term
- *Instance* of e : result of applying a substitution θ to e
 - Replace variables of e simultaneously, not sequentially

States

- *State*: a set s of ground atoms
 - The atoms represent the things that are true in one of Σ 's states
 - Only finitely many ground atoms, so only finitely many possible states



$s_1 = \{\text{attached}(\text{p1}, \text{loc1}), \text{in}(\text{c1}, \text{p1}), \text{in}(\text{c3}, \text{p1}), \text{top}(\text{c3}, \text{p1}), \text{on}(\text{c3}, \text{c1}), \text{on}(\text{c1}, \text{pallet}), \text{attached}(\text{p2}, \text{loc1}), \text{in}(\text{c2}, \text{p2}), \text{top}(\text{c2}, \text{p2}), \text{on}(\text{c2}, \text{pallet}), \text{belong}(\text{crane1}, \text{loc1}), \text{empty}(\text{crane1}), \text{adjacent}(\text{loc1}, \text{loc2}), \text{adjacent}(\text{loc2}, \text{loc1}), \text{at}(\text{r1}, \text{loc2}), \text{occupied}(\text{loc2}), \text{unloaded}(\text{r1})\}.$

States

- Literal = ground atom
- Two types of literals: positive literals, negative literals
 - `in(c1,p1)`: positive literal representing a true statement
 - `¬occupied(loc1)`: negative literal representing a true statement
- State with only positive literals => negation by failure, what is not explicitly represented is false
- State with positive and negative literals => explicit representation of true positive and negative information
 - $s1 = \{ \text{attached}(p1, \text{loc1}), \text{in}(c1, p1), \dots, \neg \text{occupied}(\text{loc1}), \neg \text{in}(c1, p2), \neg \text{in}(c3, p2), \neg \text{in}(c2, p1), \neg \text{at}(r1, \text{loc1}) \dots \}$

Operators

- *Operator*: a triple $o = (\text{name}(o), \text{precond}(o), \text{effects}(o))$
 - $\text{precond}(o)$: *preconditions*
 - literals that must be true in order to use the operator
 - $\text{effects}(o)$: *effects*
 - literals the operator will make true
 - $\text{name}(o)$: a syntactic expression of the form $n(x_1, \dots, x_k)$
 - n is an *operator symbol* - must be unique for each operator
 - $(x_1, \dots, x_k)$ is a list of every variable symbol (parameter) that appears in o
- Purpose of $\text{name}(o)$ is so we can refer unambiguously to instances of o
- Rather than writing each operator as a triple, we'll usually write it in the following format:

$\text{move}(r, l, m)$

:: robot r moves from location l to location m

precond: $\text{adjacent}(l, m), \text{at}(r, l), \neg \text{occupied}(m)$

effects: $\text{at}(r, m), \text{occupied}(m), \neg \text{occupied}(l), \neg \text{at}(r, l)$

$\text{load}(k, l, c, r)$

:: crane k at location l loads container c onto robot r

precond: $\text{belong}(k, l), \text{holding}(k, c), \text{at}(r, l), \text{unloaded}(r)$

effects: $\text{empty}(k), \neg \text{holding}(k, c), \text{loaded}(r, c), \neg \text{unloaded}(r)$

$\text{unload}(k, l, c, r)$

:: crane k at location l takes container c from robot r

precond: $\text{belong}(k, l), \text{at}(r, l), \text{loaded}(r, c), \text{empty}(k)$

effects: $\neg \text{empty}(k), \text{holding}(k, c), \text{unloaded}(r), \neg \text{loaded}$

$\text{put}(k, l, c, d, p)$

:: crane k at location l puts c onto d in pile p

precond: $\text{belong}(k, l), \text{attached}(p, l), \text{holding}(k, c), \text{top}(d, p)$

effects: $\neg \text{holding}(k, c), \text{empty}(k), \text{in}(c, p), \text{top}(c, p), \text{on}(c, d), \neg \text{top}(d, p)$

$\text{take}(k, l, c, d, p)$

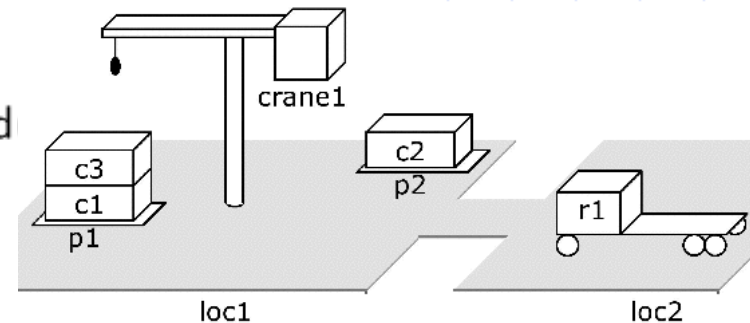
:: crane k at location l takes c off of d in pile p

precond: $\text{belong}(k, l), \text{attached}(p, l), \text{empty}(k), \text{top}(c, p), \text{on}(c, d)$

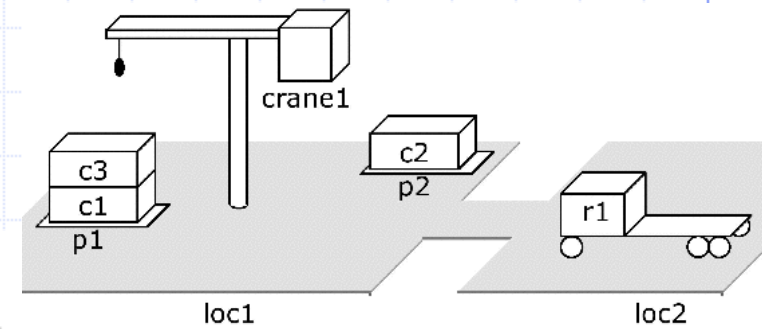
effects: $\text{holding}(k, c), \neg \text{empty}(k), \neg \text{in}(c, p), \neg \text{top}(c, p), \neg \text{on}(c, d), \text{top}(d, p)$

- **Planning domain:**
language plus
operators

- Corresponds to a
set of state-
transition systems
- Example:
operators for the
DWR domain



Actions



$\text{take}(k, l, c, d, p)$

$::$ crane k at location l takes c off of d in pile p

precond: $\text{belong}(k, l), \text{attached}(p, l), \text{empty}(k), \text{top}(c, p), \text{on}(c, d)$

effects: $\text{holding}(k, c), \neg \text{empty}(k), \neg \text{in}(c, p), \neg \text{top}(c, p), \neg \text{on}(c, d), \text{top}(d, p)$

$\text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1})$

precond: $\text{belong}(\text{crane1}, \text{loc1}), \text{attached}(\text{p1}, \text{loc1}),$
 $\text{empty}(\text{crane1}), \text{top}(\text{c3}, \text{p1}), \text{on}(\text{c3}, \text{c1})$

effects: $\text{holding}(\text{crane1}, \text{c3}), \neg \text{empty}(\text{crane1}),$
 $\neg \text{in}(\text{c3}, \text{p1}), \neg \text{top}(\text{c3}, \text{p1}), \neg \text{on}(\text{c3}, \text{c1}), \text{top}(\text{c1}, \text{p1})$

- An *action* is a ground instance (via substitution) of an operator
- Note that an action's name identifies it unambiguously
 - $\text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1})$

Notation

- Let S be a set of literals. Then
 - $S^+ = \{\text{atoms that appear positively in } S\}$
 - $S^- = \{\text{atoms that appear negatively in } S\}$;; implicitly or explicitly
- Let a be an operator or action. Then
 - $\text{precond}^+(a) = \{\text{atoms that appear positively in } a\text{'s preconditions}\}$
 - $\text{precond}^-(a) = \{\text{atoms that appear negatively in } a\text{'s preconditions}\}$
 - $\text{effects}^+(a) = \{\text{atoms that appear positively in } a\text{'s effects}\}$
 - $\text{effects}^-(a) = \{\text{atoms that appear negatively in } a\text{'s effects}\}$

$\text{take}(k, l, c, d, p)$

;; crane k at location l takes c off of d in pile p

precond: $\text{belong}(k, l), \text{attached}(p, l), \text{empty}(k), \text{top}(c, p), \text{on}(c, d)$

effects: $\text{holding}(k, c), \neg \text{empty}(k), \neg \text{in}(c, p), \neg \text{top}(c, p), \neg \text{on}(c, d), \text{top}(d, p)$

- $\text{effects}^+(\text{take}(k, l, c, d, p)) = \{\text{holding}(k, c), \text{top}(d, p)\}$
- $\text{effects}^-(\text{take}(k, l, c, d, p)) = \{\text{empty}(k), \text{in}(c, p), \text{top}(c, p), \text{on}(c, d)\}$

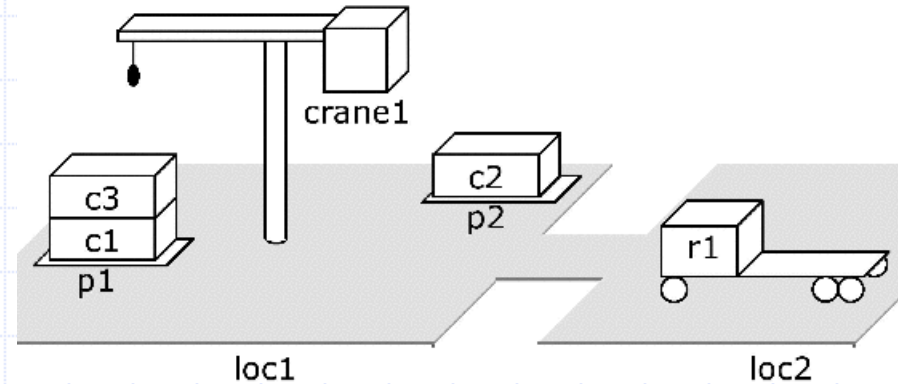
Applicability

- An action a is *applicable* to a state s if s satisfies $\text{precond}(a)$,
 - i.e., if $\text{precond}^+(a) \subseteq s$ and $\text{precond}^-(a) \cap s = \emptyset$
- An action:

$\text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1})$

precond: $\text{belong}(\text{crane1}, \text{loc1})$,
 $\text{attached}(\text{p1}, \text{loc1})$,
 $\text{empty}(\text{crane1})$, $\text{top}(\text{c3}, \text{p1})$,
 $\text{on}(\text{c3}, \text{c1})$

effects: $\text{holding}(\text{crane1}, \text{c3})$,
 $\neg \text{empty}(\text{crane1})$,
 $\neg \text{in}(\text{c3}, \text{p1})$, $\neg \text{top}(\text{c3}, \text{p1})$,
 $\neg \text{on}(\text{c3}, \text{c1})$, $\text{top}(\text{c1}, \text{p1})$



- A state it's applicable to

$s_1 = \{\text{attached}(\text{p1}, \text{loc1}), \text{in}(\text{c1}, \text{p1}),$
 $\text{in}(\text{c3}, \text{p1}), \text{top}(\text{c3}, \text{p1}), \text{on}(\text{c3}, \text{c1}),$
 $\text{on}(\text{c1}, \text{p1}), \text{attached}(\text{p2}, \text{loc1}),$
 $\text{in}(\text{c2}, \text{p2}), \text{top}(\text{c2}, \text{p2}), \text{on}(\text{c2}, \text{p2}),$
 $\text{belong}(\text{crane1}, \text{loc1}),$
 $\text{empty}(\text{crane1}),$
 $\text{adjacent}(\text{loc1}, \text{loc2}),$
 $\text{adjacent}(\text{loc2}, \text{loc1}), \text{at}(\text{r1}, \text{loc2}),$
 $\text{occupied}(\text{loc2}), \text{unloaded}(\text{r1})\}$

Executing an Applicable Action

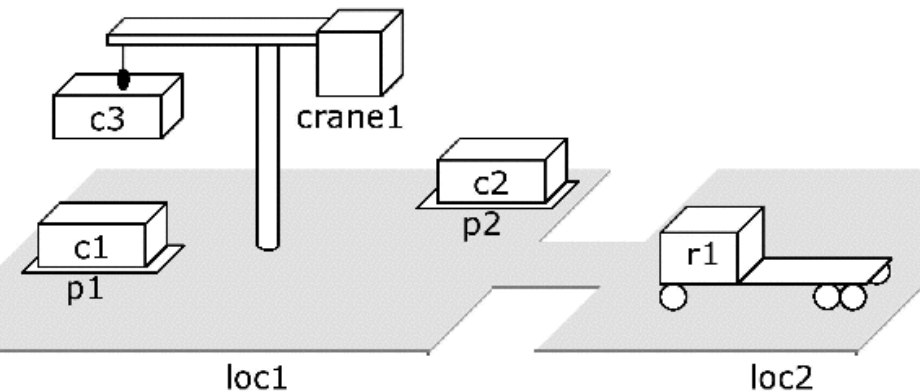
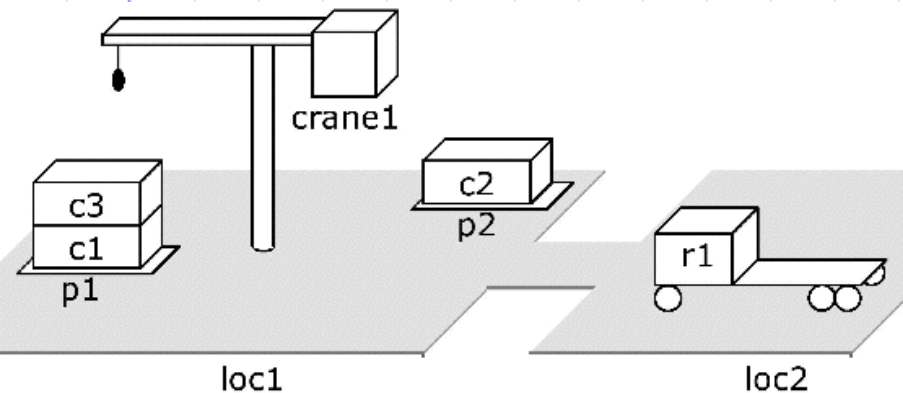
- Remove a 's negative effects, and add a 's positive effects

$$\gamma(s, a) = (s - \text{effects}^-(a)) \cup \text{effects}^+(a)$$

take(crane1, loc1, c3, c1, p1)

precond: belong(crane1, loc1),
attached(p1, loc1),
empty(crane1),
top(c3, p1), on(c3, c1)

effects: holding(crane1, c3),
 $\neg$ empty(crane1),
 $\neg$ in(c3, p1), $\neg$ top(c3, p1),
 $\neg$ on(c3, c1), top(c1, p1)



$s_2 = \{\text{attached}(p1, \text{loc1}), \text{in}(c1, p1), \text{in}(c3, p1),$
 ~~$\text{top}(c3, p1), \text{on}(c3, c1)$~~ , $\text{on}(c1, p1),$
 $\text{attached}(p2, \text{loc1}), \text{in}(c2, p2), \text{top}(c2, p2),$
 $\text{on}(c2, p2), \text{belong}(\text{crane1}, \text{loc1}),$
 ~~$\text{empty}(\text{crane1})$~~ , $\text{adjacent}(\text{loc1}, \text{loc2}),$
 $\text{adjacent}(\text{loc2}, \text{loc1}), \text{at}(r1, \text{loc2}),$
 $\text{occupied}(\text{loc2}, \text{unloaded}(r1),$
 $\text{holding}(\text{crane1}, c3), \text{top}(c1, p1)\}$

Planning Problems

- Given a planning domain (language L , operators O)
 - *Statement* of a planning problem: a triple $P=(O,s_0,g)$
 - O is the collection of operators
 - s_0 is a state (the initial state)
 - g is a set of literals (the goal formula)
 - Planning problem: $\mathcal{P} = (\Sigma,s_0,S_g)$
 - s_0 = initial state
 - S_g = set of goal states
 - $\Sigma = (S,A,\gamma)$ is a state-transition system
 - $S = \{\text{all sets of ground atoms in } L\}$
 - $A = \{\text{all ground instances of operators in } O\}$
 - γ = the state-transition function determined by the operators
- I'll often say "planning problem" when I mean the statement of the problem

Plans and Solutions

- *Plan*: any sequence of actions $\sigma = \langle a_1, a_2, \dots, a_n \rangle$ such that each a_i is an instance of an operator in O
- The plan is a *solution* for $P=(O, s_0, g)$ if it is executable and achieves g
 - i.e., if there are states $s_0, s_1, \dots, s_n$ such that
 - $\gamma(s_0, a_1) = s_1$
 - $\gamma(s_1, a_2) = s_2$
 - ...
 - $\gamma(s_{n-1}, a_n) = s_n$
 - s_n satisfies g

Example

- Let $P_1 = (O, s_1, g_1)$, where
 $O = \{\text{the five DWR operators given earlier}\}$

$$g_1 = \{\text{loaded}(r1, c3), \text{at}(r1, \text{loc2})\}$$

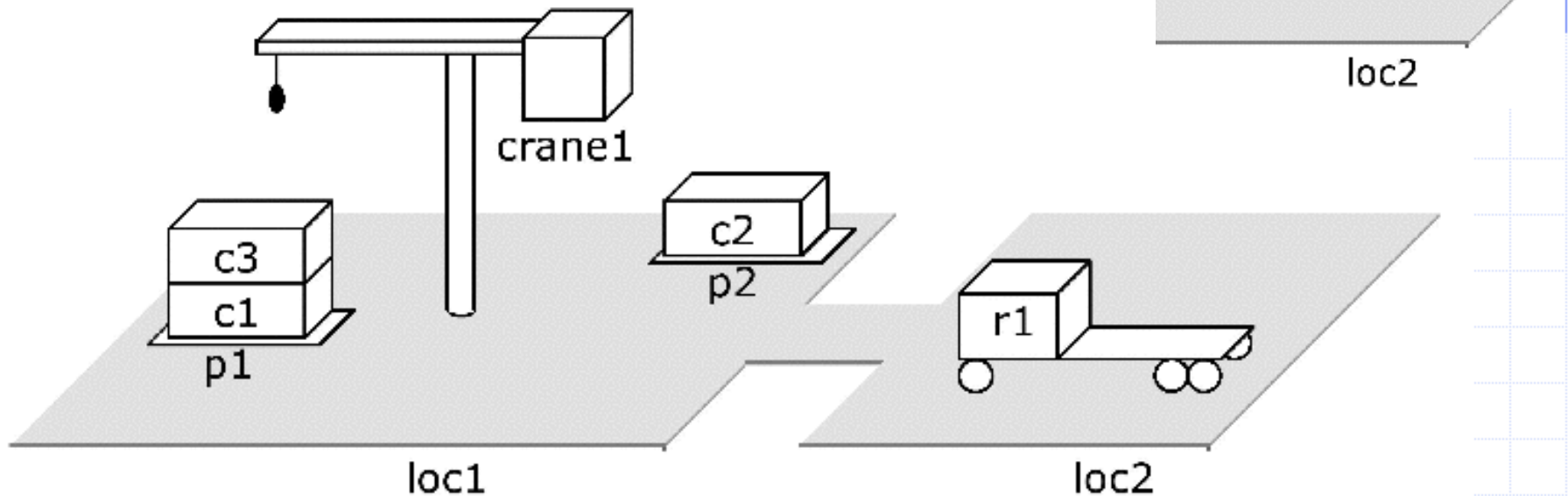
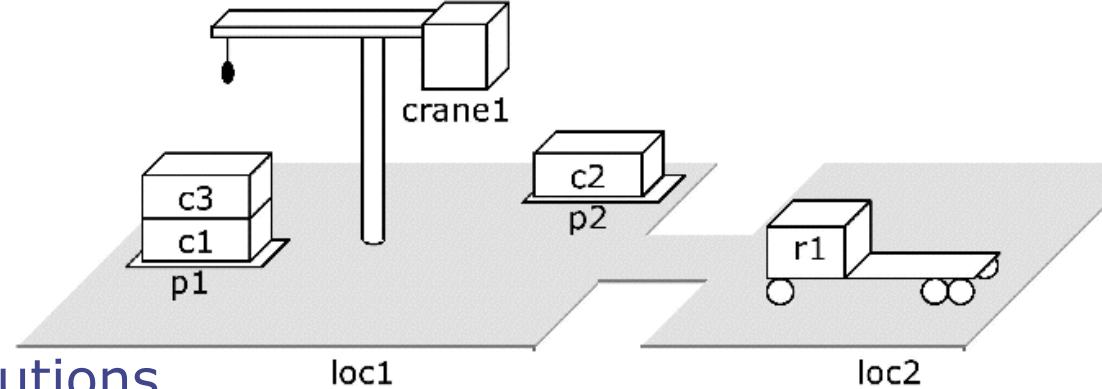


Figure 2.2: The DWR state $s_1 = \{\text{attached}(p1, \text{loc1}), \text{in}(c1, p1), \text{in}(c3, p1), \text{top}(c3, p1), \text{on}(c3, c1), \text{on}(c1, \text{pallet}), \text{attached}(p2, \text{loc1}), \text{in}(c2, p2), \text{top}(c2, p2), \text{on}(c2, \text{pallet}), \text{belong}(\text{crane1}, \text{loc1}), \text{empty}(\text{crane1}), \text{adjacent}(\text{loc1}, \text{loc2}), \text{adjacent}(\text{loc2}, \text{loc1}), \text{at}(r1, \text{loc2}), \text{occupied}(\text{loc2}), \text{unloaded}(r1)\}$.

Example, continued



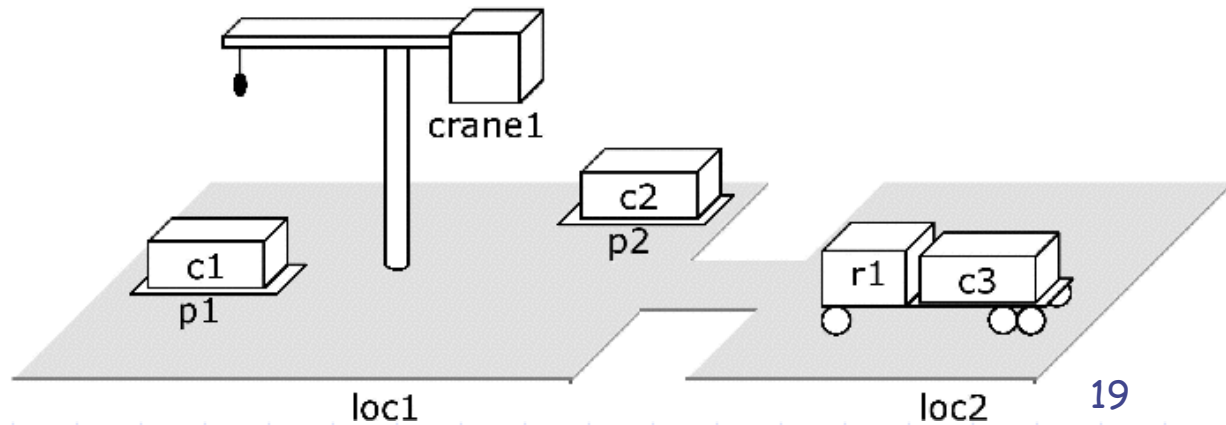
- P_1 has infinitely many solutions
- Here are three of them:

$\langle \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}),$
 $\text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

$\langle \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}),$
 $\text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

$\langle \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}),$
 $\text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

- They each produce this state:



Example, continued

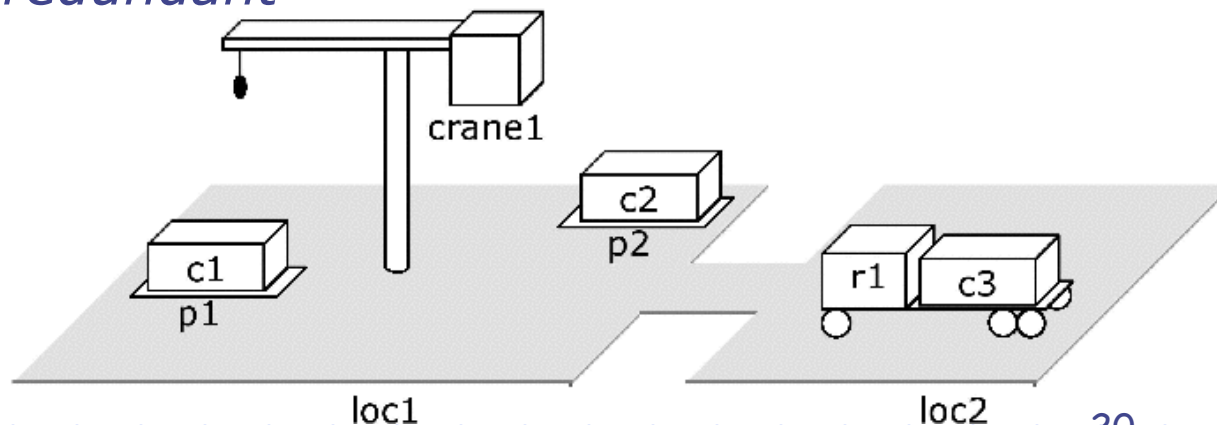
- The first one is *redundant*
 - Can remove actions and still have a solution

$\langle \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}), \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

$\langle \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

$\langle \text{move}(\text{r1}, \text{loc2}, \text{loc1}), \text{take}(\text{crane1}, \text{loc1}, \text{c3}, \text{c1}, \text{p1}), \text{load}(\text{crane1}, \text{loc1}, \text{c3}, \text{r1}), \text{move}(\text{r1}, \text{loc1}, \text{loc2}) \rangle$

- The 2nd and 3rd are *irredundant*
- They also are *shortest*
 - No shorter solutions exist



State-Variable Representation

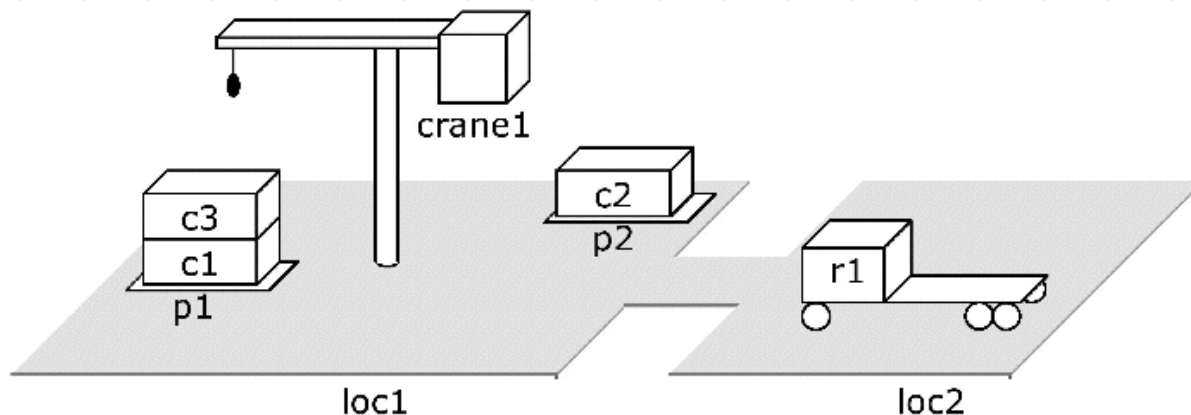
- Use ground atoms for properties that do not change, e.g., $\text{adjacent}(\text{loc1}, \text{loc2})$
- For properties that can change, assign values to *state variables*
 - Like fields in a record structure
- Classical and state-variable representations take similar amounts of space
 - Each can be translated into the other in low-order polynomial time

$\text{move}(r, l, m)$

;; robot r at location l moves to an adjacent location m

precond: $\text{rloc}(r) = l, \text{adjacent}(l, m)$

effects: $\text{rloc}(r) \leftarrow m$

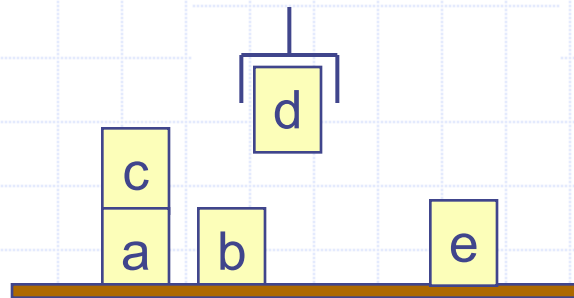


$s_1 = \{\text{top}(p1)=c3,$
 $\text{cpos}(c3)=c1,$
 $\text{cpo}(c1)=\text{pallet},$
 $\text{holding}(\text{crane1})=\text{nil},$
 $\text{rloc}(r1)=\text{loc2},$
 $\text{loaded}(r1)=\text{nil}, \text{21.}\}$

Example: The Blocks World

- Infinitely wide table, finite number of children's blocks
- Ignore where a block is located on the table
- A block can sit on the table or on another block
- There's a robot gripper that can hold at most one block
- Want to move blocks from one configuration to another
 - e.g.,

initial state



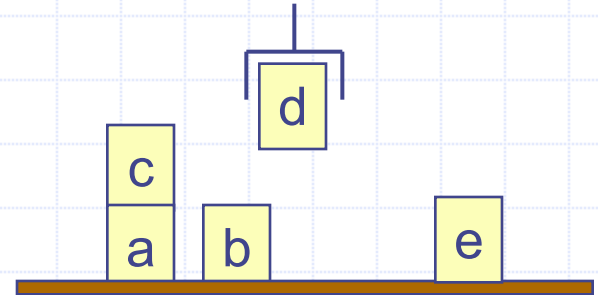
goal



- Like a special case of DWR with one location, one crane, some containers, and many more piles than you need
- I'll give classical and state-variable formulations
 - For the case where there are five blocks

Classical Representation: Symbols

- Constant symbols:
 - The blocks: a, b, c, d, e
- Predicates:
 - `ontable(x)` - block x is on the table
 - `on(x,y)` - block x is on block y
 - `clear(x)` - block x has nothing on it
 - `holding(x)` - the robot hand is holding block x
 - `handempty` - the robot hand isn't holding anything



Classical Operators

unstack(x,y)

Precond: $\text{on}(x,y)$, $\text{clear}(x)$, handempty

Effects: $\neg\text{on}(x,y)$, $\neg\text{clear}(x)$, $\neg\text{handempty}$,
 $\text{holding}(x)$, $\text{clear}(y)$

stack(x,y)

Precond: $\text{holding}(x)$, $\text{clear}(y)$

Effects: $\neg\text{holding}(x)$, $\neg\text{clear}(y)$,
 $\text{on}(x,y)$, $\text{clear}(x)$, handempty

pickup(x)

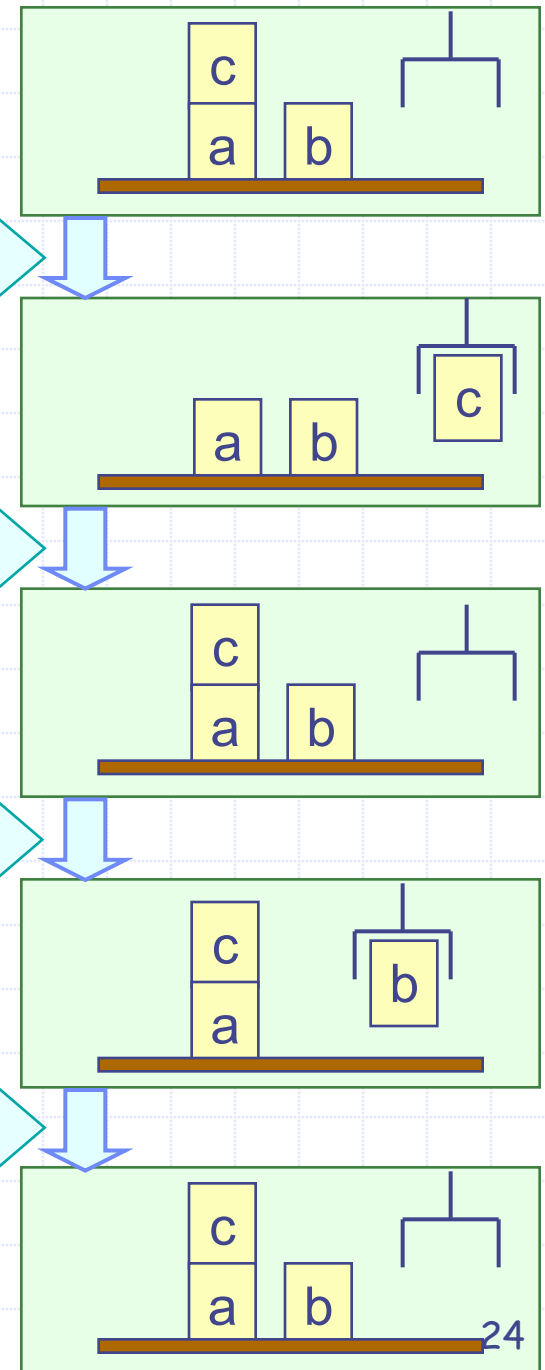
Precond: $\text{ontable}(x)$, $\text{clear}(x)$, handempty

Effects: $\neg\text{ontable}(x)$, $\neg\text{clear}(x)$,
 $\neg\text{handempty}$, $\text{holding}(x)$

putdown(x)

Precond: $\text{holding}(x)$

Effects: $\neg\text{holding}(x)$, $\text{ontable}(x)$,
 $\text{clear}(x)$, handempty



State-Variable Representation: Symbols

- Constant symbols:

a, b, c, d, e

of type block

0, 1, table, nil

of type other

- State variables:

$\text{pos}(x) = y$

if block x is on block y

$\text{pos}(x) = \text{table}$

if block x is on the table

$\text{pos}(x) = \text{nil}$

if block x is being held

$\text{clear}(x) = 1$

if block x has nothing on it

$\text{clear}(x) = 0$
on it

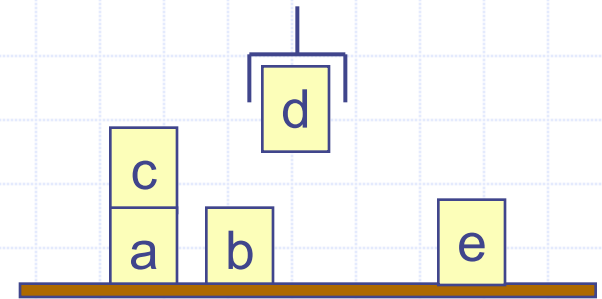
if block x is being held or has another block

$\text{holding} = x$

if the robot hand is holding block x

$\text{holding} = \text{nil}$

if the robot hand is holding nothing

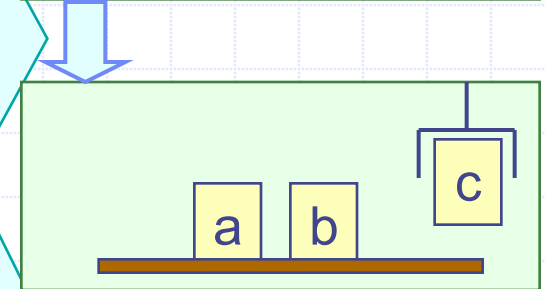
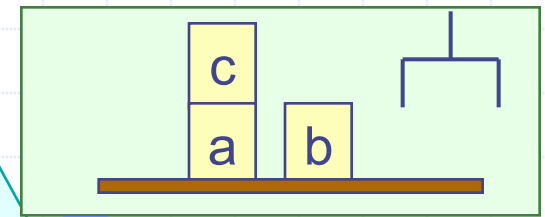


State-Variable Operators

unstack(x : block, y : block)

Precond: $\text{pos}(x)=y$, $\text{clear}(y)=0$, $\text{clear}(x)=1$, $\text{holding}=\text{nil}$

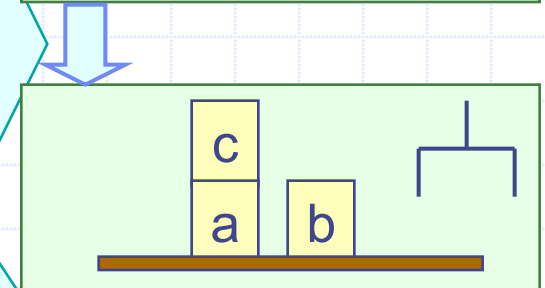
Effects: $\text{pos}(x)=\text{nil}$, $\text{clear}(x)=0$, $\text{holding}=x$, $\text{clear}(y)=1$



stack(x : block, y : block)

Precond: $\text{holding}=x$, $\text{clear}(x)=0$, $\text{clear}(y)=1$

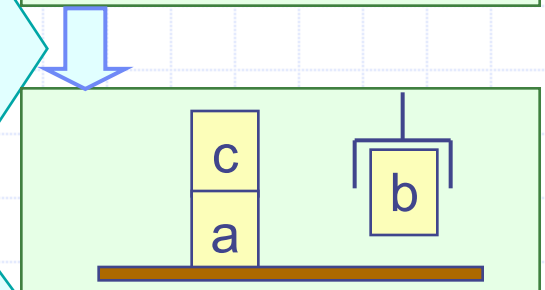
Effects: $\text{holding}=\text{nil}$, $\text{clear}(y)=0$, $\text{pos}(x)=y$, $\text{clear}(x)=1$



pickup(x : block)

Precond: $\text{pos}(x)=\text{table}$, $\text{clear}(x)=1$, $\text{holding}=\text{nil}$

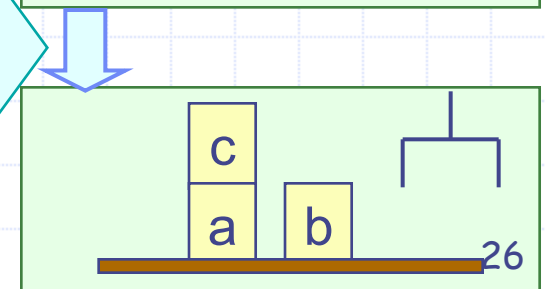
Effects: $\text{pos}(x)=\text{nil}$, $\text{clear}(x)=0$, $\text{holding}=x$



putdown(x : block)

Precond: $\text{holding}=x$

Effects: $\text{holding}=\text{nil}$, $\text{pos}(x)=\text{table}$, $\text{clear}(x)=1$



Comparison

- Classical representation
 - The most popular for classical planning, partly for historical reasons
- State-variable representation
 - Equivalent to classical representation in expressive power
 - Less natural for logicians, more natural for engineers
 - Useful in non-classical planning problems as a way to handle numbers, functions, time

PDDL.

Planning Domain Description Language.

- We will only use Classical Representation
- Examples: Blocks-world, Hanoi towers
- Two files: domain file and problem file
- Domain file: predicates, operators
- Problem file: problem objects, initial state
- PDDL BNF syntax provided

PDDL. Blocks-world. Domain file (I)

- Objects in the domain: blocks, table, robot-arm
- Properties of the objects

```
(define (domain blockword)
  (:predicates
    (clear ?x)
    (on-table ?x)
    (arm-empty)
    (holding ?x)
    (on ?x ?y))
```

PDDL. Blocks-world. Domain file (II)

```
(:action pickup
:parameters (?ob)
:precondition
    (and (clear ?ob) (on-table ?ob) (arm-empty))
:effect
    (and (holding ?ob) (not (clear ?ob))
        (not (on-table ?ob)) (not (arm-empty))))
```

```
(:action putdown
:parameters (?ob)
:precondition (and (holding ?ob))
:effect
    (and (clear ?ob) (arm-empty) (on-table ?ob)
        (not (holding ?ob))))
```

PDDL. Blocks-world. Domain file (III)

```
(:action stack
:parameters (?ob ?underob)
:precondition
    (and (clear ?underob) (holding ?ob))
:effect
    (and (clear ?ob) (arm-empty) (on ?ob ?underob)
        (not (holding ?ob)) (not (clear ?underob))))
```

```
(:action unstack
:parameters (?ob)
:precondition (and (on ?ob ?underob) (clear ?ob)
    (arm-empty))
:effect (and (holding ?ob) (clear ?underob)
    (not (on ?ob ?underob)) (not (clear ?ob))
    (not (arm-empty))))
)
```

PDDL. Blocks-world. Problem file

```
(define (problem tower6)
  (:domain blocksworld)
  (:objects a b c d e f)

  (:init (on-table a) (on-table b) (on-table c)
          (on-table d) (on-table e) (on-table f)
          (clear a) (clear b) (clear c) (clear d)
          (clear e) (clear f) (arm-empty))

  (:goal (and (on a b) (on b c) (on c d) (on d e)
              (on e f))))
```


PDDL. Blocks-world (typing-I)

- Using 'typing': define types of objects, an object hierarchy

```
(define (domain blocksworld)
  (:requirements :typing)
  (:types block)
  (:predicates (on ?x - block ?y - block)
               (ontable ?x - block)
               (clear ?x - block)
               (handempty)
               (holding ?x - block)
  )
)
```

PDDL. Blocks-world (typing-II)

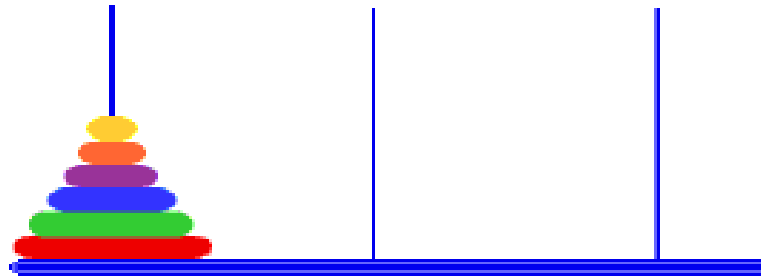
```
(:action stack
  :parameters (?ob - block ?underob - block)
  :precondition
    (and (clear ?underob) (holding ?ob))
  :effect
    (and (clear ?ob) (arm-empty) (on ?ob ?underob)
      (not (holding ?ob)) (not (clear ?underob))))
```

```
(define (problem tower6)
  (:domain blocksworld)
  (:objects a b c d e f - block)

  (:init . . .)

  (:goal . . .))
```

PDDL. Hanoi towers.



- Three disks: large (L), medium (M), small (S)
- Three pegs: peg1 (P1), peg2 (P2), peg3 (P3)
- Two predicates: (at <disk> <disk|peg>)
(clear <disk|peg>)

PDDL. Hanoi towers (domain I).

```
(define (domain hanoi)
  (:requirements :strips :typing :equality)
  (:types disk peg)
  (:predicates (at ?x - disk ?y - (either disk peg))
               (clear ?x - (either disk peg)))

  (:action move-large
    :parameters (?x - peg ?y - peg)
    :precondition (and (at L ?x) (clear L) (clear ?y))
    :effect
      (and (not (at L ?x)) (at L ?y)
           (not (clear ?y)) (clear ?x)))
```

PDDL. Hanoi towers (domain II).

```
(:action move-medium
  :parameters (?x - (either peg disk) ?y - (either disk peg))
  :precondition (and (at M ?x) (clear M)
                     (clear ?y) (not (= ?y S))))
  :effect (and (not (at M ?x)) (at M ?y) (not (clear ?y))
               (clear ?x)))

(:action move-small
  :parameters (?x - (either peg disk) ?y - (either disk peg))
  :precondition (and (at S ?x) (clear S) (clear ?y))
  :effect
    (and (not (at S ?x)) (at S ?y)
          (not (clear ?y)) (clear ?x)))
```

PDDL. Hanoi towers (problem).

```
(define (problem probhanoi1)
  (:domain hanoi)
  (:objects L M S - disk
            P1 P2 P3 - peg)

  (:init (at S M) (at M L) (at L P1)
         (clear S) (clear P2) (clear P3))

  (:goal (and (at S M) (at M L) (at L P3)))
)
```

PDDL. Hanoi towers (a different encoding)

```
(define (domain hanoi)
  (:requirements :strips :typing :equality
                 :negative-preconditions)
  (:types disk peg)
  (:predicates (at ?x - disk ?y - (either disk peg))
               (clear ?x - (either disk peg)))

  (:action move-large
    :parameters (?x - peg ?y - peg)
    :precondition (and (at L ?x) (clear L)
                       (not (at M ?y)) (not (at S ?y)))
    :effect (and (not (at L ?x)) (at L ?y)
                 (not (clear ?y)) (clear ?x)))
```

PDDL. Hanoi towers (a different encoding with only one operator)

```
(define (domain hanoi)
  (:requirements :strips :typing)
  (:types disk peg)
  (:predicates (at ?x - disk ?y - (either disk peg))
               (clear ?x - (either disk peg))
               (smaller ?x - disk ?y - (either disk peg)))

  (:action move-disk
    :parameters (?disk - disk ?from - (either disk peg)
                 ?new-below - (either disk peg))
    :precondition (and (at ?disk ?from)
                       (clear ?disk) (clear ?new-below)
                       (smaller ?disk ?new-below))
    :effect (and (at ?disk ?new-below) (clear ?from)
                 (not (clear ?new-below))
                 (not (at ?disk ?from))))
)
```


PDDL. Hanoi towers (a different encoding with only one operator)



```
(define (problem probhanoi1)
(:domain hanoi)
(:objects L M S - disk
          P1 P2 P3 - peg)

(:init (at S M) (at M L) (at L P1) (clear S) (clear P2)
        (clear P3) (smaller S M) (smaller S L) (smaller M L)
        (smaller S P1) (smaller S P2) (smaller S P3)
        (smaller M P1) (smaller M P2) (smaller M P3)
        (smaller L P1) (smaller L P2) (smaller L P3))

(:goal (and (at S M) (at M L) (at L P3)))
)
```